

FinalSay — Prototype Presentation Report

"Verify. Don't just forward."

Team

Ragini · Jatin Goyal · Vaishnavi Singh · Madhav

Instructor

Prof. Sukhpal Singh

Prototype site

goyaljatin870.github.io/FinalSay-prototype

Presentation window

Mon 14 – Fri 18 Sept 2026 (scheduled lab)

Coverage check against the brief — read this first

Use Case Diagrams: 1 delivered / 4 required · Sequence Diagrams: 1 delivered / 5 required · Class Diagram: 1 delivered / 1 required

MET

This report uses the three diagrams you have ready (one Use Case, one Sequence, one Class) instead of the full set the mail asks for. Section 7 below lays out a concrete, actor-grounded split for the three outstanding Use Case Diagrams and four outstanding Sequence Diagrams — treat that as a punch list, not a finished deliverable. Your lab window is already open (today is 17 Sept); close this gap before you present, not during.

01 The problem, the users, and why we picked it

Institutional notices — circulars, office memos, admissions or examination updates — travel through screenshots, forwarded chat threads, and copy-pasted text long after they leave the issuing office's own page. Once a notice is separated from its source, a reader has no quick way to tell whether it still matches what the institution currently says, has been quietly revised, or has been contradicted by a newer notice. FinalSay exists to close that gap: it takes a notice a student has received and checks it against the institution's own published record.

Users

ACTOR	ROLE	WHAT THEY NEED FROM THE SYSTEM
Student	Primary	Submit a notice they received and find out if it's still accurate, contradicted, or superseded
Reviewer / Annotator	Primary	Resolve the cases the automated pipeline can't confidently classify
Administrator	Primary	Maintain the list of trusted institution sources and the official notice repository the system checks against
Institution Issuer	Secondary	Publish the official notices that become the system's source of truth

Why this problem

The team's original proposal, NETRA — an Indic-script accessibility auditor for institutional websites — was turned down for insufficient user validation. The team ran a same-day ideation session, stress-tested five candidate ideas against a seven-criteria rubric, ruled out ThrottleWatch on discovering the peer-reviewed Wehe tool covered the same ground, and converged on NoticeProof, later renamed FinalSay. Prior-art checks (ContraGen, and NLI/LLM-hybrid contradiction-detection research) confirmed the space was real but not already solved off-the-shelf. The case for the problem itself is backed by PIB Fact Check data and a seven-institution incident timeline spanning 2021–2026.

02 Proposed solution & the user journey implemented

A student logs in, submits a notice as a PDF, image, or plain text, and FinalSay runs it through a fixed pipeline: extract the text if it wasn't already text, pull the matching official notice(s) from a registered institution source, run a semantic comparison between the two, and classify the relationship as **Consistent**, **Contradictory**, **Superseded**, or **Unresolved**. The system then writes a plain-language explanation, records a verification hash for auditability, and hands the student a result they can act on — with the option to drill into the supporting evidence or check the record's integrity.

1. Student logs in and submits a notice (PDF / image / text).
2. If the notice isn't already text, it's sent for OCR extraction.
3. The system retrieves the relevant official notice(s) from the matching institution source.
4. Submitted text and official text are compared for semantic similarity.
5. The relationship is classified — Consistent, Contradictory, Superseded, or Unresolved.
6. An explanation and supporting evidence are generated.
7. A verification hash is recorded for integrity.
8. The result, evidence, and integrity info are shown to the student, who can optionally inspect the evidence or check integrity separately.

03 Live demo script

One complete flow, start to finish, using the actual hosted site and prepared test data — no fixed sample outputs.

Before you go up

- Open the site in a signed-out / incognito browser and confirm it loads with no access request.
- Have one test notice ready that you expect to come back **Consistent**, and ideally a second that comes back **Contradictory** or **Superseded** — a single happy-path demo doesn't show the classifier actually discriminating.
- Confirm every external call you're about to demo (OCR, source retrieval, NLI comparison, hash recording) is actually live, not stubbed — say so out loud if any leg is mocked rather than let it pass as real.

During the walkthrough

- 1 Log in as a student on the real site.
- 2 Submit the prepared test notice.
- 3 Narrate the pipeline live as it runs — extraction (if triggered), retrieval of the official notice, comparison, classification.
- 4 Show the result screen and read out the classification and explanation the system actually produced.
- 5 Open "View Evidence" and "Check Integrity" to show the supporting record and the hash.
- 6 Repeat steps 2-5 with the second test notice to show a different classification outcome.

04 Working integration

From the class and sequence diagrams, these are the pieces that have to actually talk to each other for the demo to count as "integrated" rather than a set of disconnected screens:

COMPONENT	ROLE IN THE PIPELINE	STATUS (FILL IN)
Web / App UI	Login, notice submission, result & evidence display	—
Backend API server	Orchestrates the verification pipeline end to end	—
OCR Service	Extracts text from PDF / image submissions	—
Official Notice Source	Supplies the official notice(s) to compare against	—
NLI / LLM Service	Semantic comparison + relationship classification	—
Blockchain / Integrity Service	Records and verifies the result hash	—
Data layer (Notice, Document, Verification, Evidence)	Persists submissions, results, and evidence	—

Note: the mail is explicit that a fixed sample result doesn't count as a demonstration. For anything you mark as live above, be ready to point at the code that makes the call, not just the UI that displays the result.

05 What's complete, what's limited, what's next

FEATURE	STATUS	NOTES
Register / Login	<i>TODO</i>	—

Submit notice + OCR extraction	<i>TODO</i>	—
Retrieve official notice(s)	<i>TODO</i>	—
Semantic comparison + classification	<i>TODO</i>	—
Evidence generation	<i>TODO</i>	—
Integrity hash recording / verification	<i>TODO</i>	—
Reviewer queue & annotation	<i>TODO</i>	—
Admin: source & repository management	<i>TODO</i>	—
Institution Issuer publish flow	<i>TODO</i>	—

Rows are seeded from the class/sequence diagrams so nothing gets missed — fill Status as Complete, Partial, or Missing against what's actually in the repo, then use this table as your "what's next" slide.

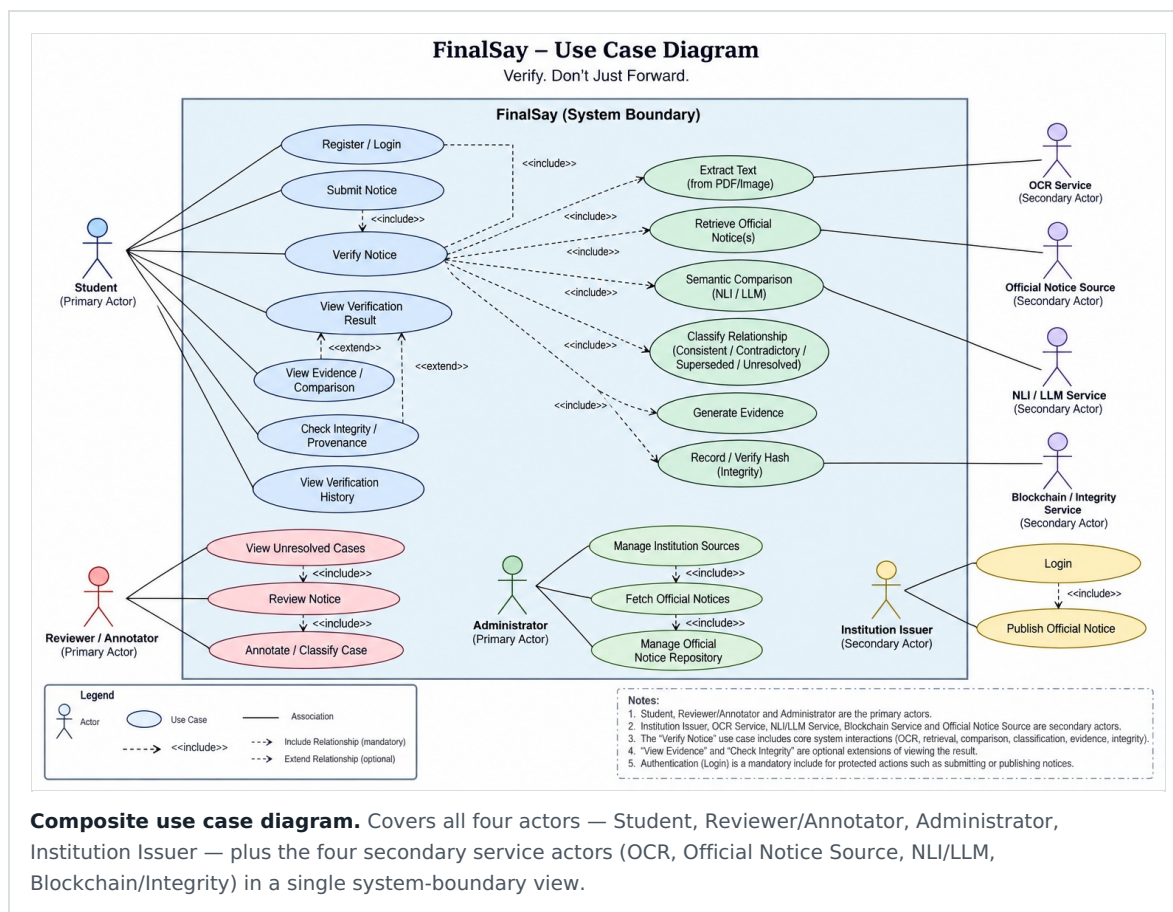
06 Individual contributions

The mail asks for each member's contribution backed by repository evidence — commits, issues, tests, docs, or journal entries — and warns that any member may be asked to locate their own code live. Fill this in from the repo, not from memory of who said they'd do what.

MEMBER	AREA(S) WORKED ON	EVIDENCE (COMMITTS / ISSUES / PRS / JOURNAL)
Ragini	—	—
Jatin Goyal	—	—
Vaishnavi Singh	—	—
Madhav	—	—

07 Diagrams

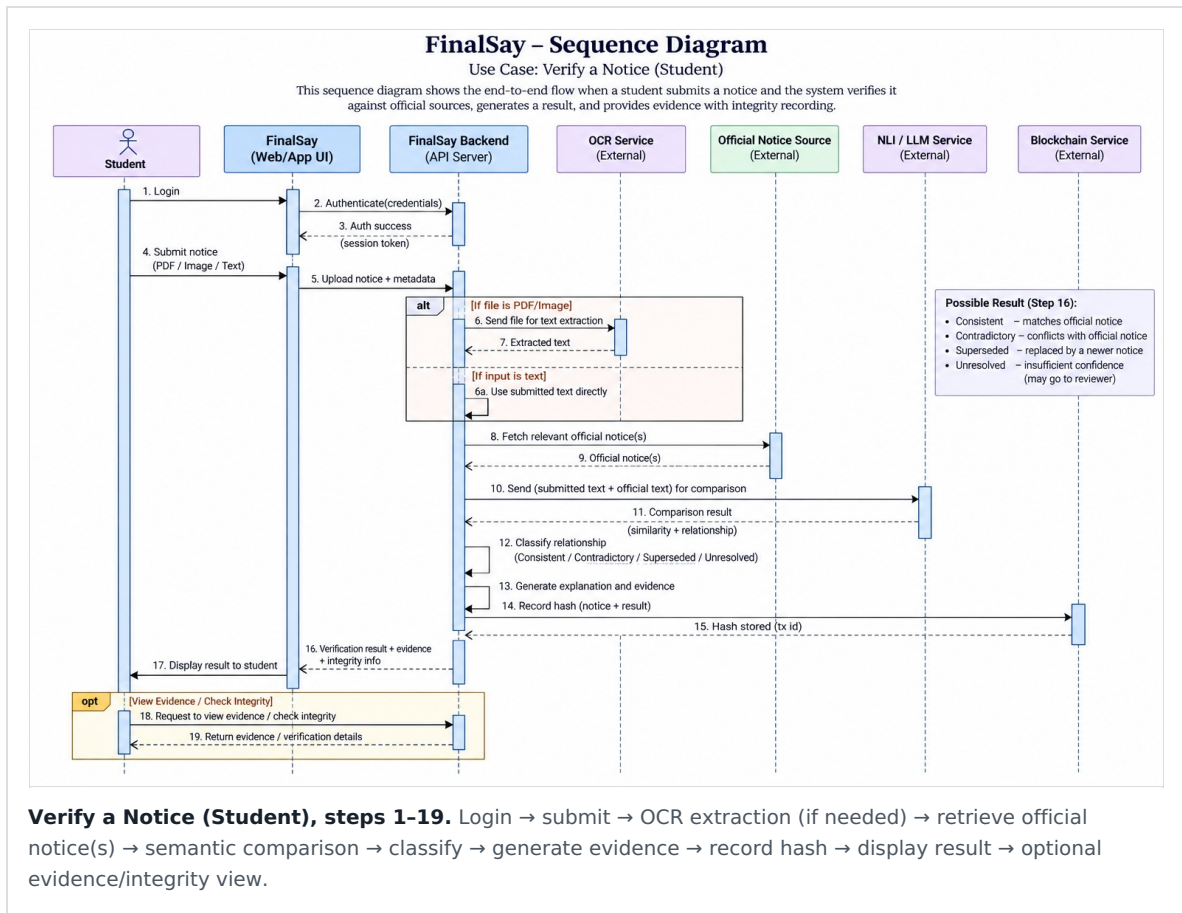
7.1 Use Case Diagram — delivered 1 / 4



This one diagram already covers every actor at once, which is exactly why it can't stand in for four. Confirm with the professor whether "4 Use Case Diagrams" means one per actor or one per module — the cleanest actor-based split, using what's already defined here:

#	SUGGESTED FOCUS	SCOPE
UCD-1	Student — delivered above, can be trimmed to just this actor	Register/Login, Submit Notice, Verify Notice (+ its includes), View Result, View Evidence, Check Integrity, View History
UCD-2	Reviewer / Annotator	View Unresolved Cases, Review Notice, Annotate/Classify Case
UCD-3	Administrator	Manage Institution Sources, Fetch Official Notices, Manage Official Notice Repository
UCD-4	Institution Issuer	Login, Publish Official Notice

7.2 Sequence Diagram — delivered 1 / 5



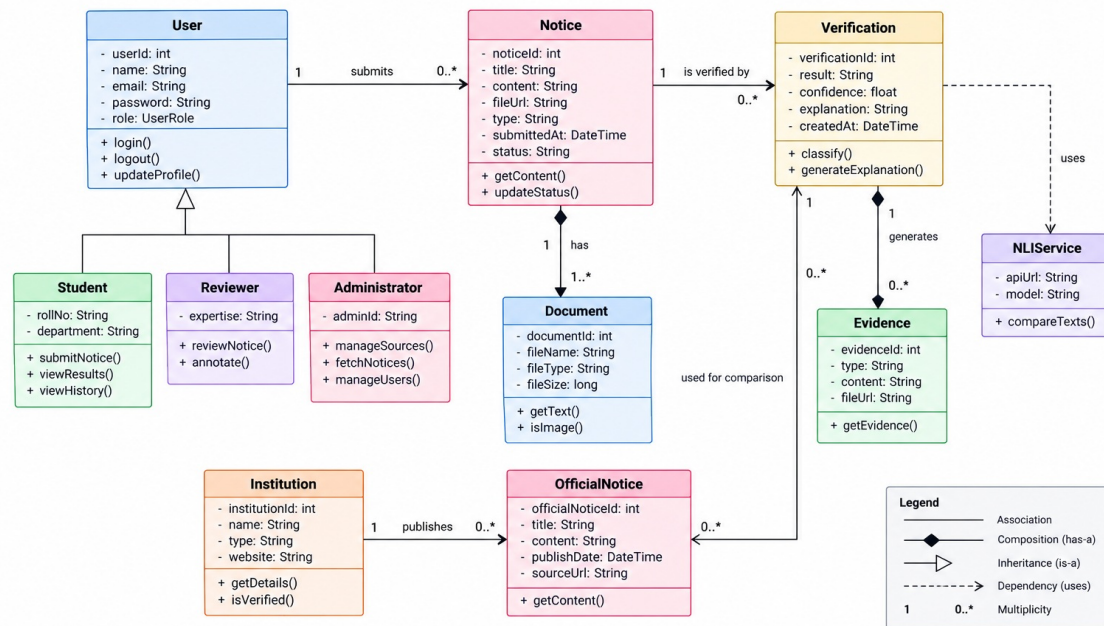
Four more flows to diagram, one per outstanding actor path plus the failure/auth case:

#	FLOW
SD-2	Reviewer resolves an Unresolved case (View Unresolved Cases → Review Notice → Annotate/Classify Case)
SD-3	Administrator manages sources and fetches official notices (Manage Institution Sources → Fetch Official Notices → Manage Repository)
SD-4	Institution Issuer publishes a notice (Login → Publish Official Notice)
SD-5	Registration/Login, or a failure path worth showing explicitly — e.g. OCR failure, or a low-confidence result routed to a reviewer instead of shown as Unresolved outright

7.3 Class Diagram (Simplified) — delivered 1 / 1

FinalSay – Class Diagram (Simplified)

Key classes and relationships for verifying institutional notices.



Key classes and relationships, simplified pass. A User hierarchy (Student, Reviewer, Administrator) submits Notice objects composed of one or more Documents; each Notice is verified through a Verification, which calls out to an NLIService for comparison and generates Evidence. Institution publishes OfficialNotice records, which feed into Verification for comparison.

Check before you present: this simplified diagram drops OCRService, BlockchainService, and the InstitutionIssuer actor that Sections 2–4 and the delivered Use Case / Sequence diagrams (7.1–7.2) describe as part of the pipeline. Either this class diagram is intentionally scoped to the notice-verification core (fine — say so explicitly to the professor), or it needs those three back in. Don't let the diagram set silently disagree with itself.

Built from the three diagrams supplied, the professor's mail (14–18 Sept 2026 prototype window), and the project's own proposal notes. This is a working aid for assembling your presentation and site content — the course page separately lists a LaTeX-typeset "Prototype Stage Progress Report" as an MST deliverable (template TBA in the course repo at time of writing); check with the professor whether that's active this cycle, since it's distinct from the presentation covered here.